

## **GIGAPAXOS**

### **Overview:**

MyDBReplicableAppGP is a replicated state machine (RSM) built on top of GigaPaxos to provide fault-tolerant database operations. It works with Cassandra to persist data and ensures that all replicas execute operations in a consistent, deterministic order. The main goals of system are to prevent duplicate execution of requests, preserve the correct order of operations, support regular checkpointing, and allow full recovery of state in case of failures.

### **State Management::**

The class uses a ConcurrentHashMap called `executedRequests` to track executed request IDs and prevent duplicate execution. A separate `requestLog` list maintains the order of recent requests up to a defined maximum (`MAX_LOG_SIZE`) for memory management. The system also maintains `latestExecutedSeq` to track the highest sequence number executed. Thread safety is ensured using concurrent data structures and synchronized blocks where necessary, especially during logging and checkpoint operations.

### **Database Integration:**

Cassandra serves as the persistent storage backend for the system. When the application starts, it automatically creates the keyspace if it doesn't already exist and ensures that the grade table is available. The table is designed with a simple schema: an integer primary key (`id`) and a list of integers (`events`). The `executeSQL()` method handles executing queries and includes a built-in recovery mechanism. If a query fails because the table doesn't exist, the system automatically creates the table and retries the query. This design minimizes the need for manual intervention and helps ensure that all state changes are reliably applied.

### **Request Handling:**

Requests arrive as `RequestPacket` instances. SQL statements are extracted either from a JSON "QV" field or directly from the packet value. Before execution, the system checks for duplicates to maintain idempotency. Sequence numbers are parsed from SQL patterns of the form `events+ [<seq>]` to maintain execution order and update `latestExecutedSeq`. Executed requests are logged and old entries are removed when the log exceeds `MAX_LOG_SIZE`.

### **Checkpointing:**

Checkpointing captures the entire system state in a serializable object called `CheckpointState`. This object contains the keyspace name, timestamp, latest executed sequence, executed request IDs, and the complete database state. Checkpoints are serialized to a Base64 string to enable storage or replication. Periodic checkpoints are triggered based on a fixed interval, and on-demand checkpointing is supported as well. The checkpoint mechanism ensures that both in-memory and persistent state can be fully restored after a failure.

### **Restoration:**

The `restore()` method decodes Base64 checkpoint strings and restores all in-memory data structures and Cassandra state. Invalid or empty checkpoints are handled gracefully, starting the system in a fresh state if necessary. This ensures that the system can always recover from crashes or corrupted state, maintaining consistency across replicas

### **Testing and Results:**

All tests are passed

## ZOOKEEPER

### Overview:

MyDBFaultTolerantServerZK is a fault-tolerant server built on top of a datastore that uses ZooKeeper for coordination and Cassandra for storage. The server processes client requests stored as sequential znodes in ZooKeeper, executes them against a Cassandra grade table, and records progress as lightweight checkpoints in ZooKeeper. The design prioritizes sequential, deterministic execution of requests, crash recovery with minimal lost work, and pragmatic simplicity so it integrates cleanly with the provided testing harness.

### State management:

The server tracks execution progress with a long `lastProcessedSeq`, which holds the highest sequential request index that has been applied to Cassandra. An `AtomicLong` operation counter controls periodic checkpointing, and a concurrent `processedRequests` set records which ZooKeeper request nodes have already been executed to avoid duplicate work if the same znode is encountered again. A recovering flag prevents concurrent processing while recovery is in progress. Synchronization is applied conservatively around the request-processing loop to ensure only one thread advances sequence numbers and applies requests at a time.

### ZooKeeper usage:

ZooKeeper is initialized at `localhost:2181` and the server uses a compact path layout rooted at `/ftdb`. Client requests are created under `/ftdb/requests` as persistent sequential nodes (`req-0000000001`, etc.). Checkpoints are stored under `/ftdb/checkpoint/<seq>` as binary blobs containing a serialized snapshot of the grade table, and the last checkpoint sequence is mirrored in `/ftdb/last_seq`. The server watches the requests path and reads children ordered lexicographically to determine processing order.

### Database Integration:

On startup the server connects to a Cassandra keyspace named after the server ID and ensures a simple grade table exists with the schema (`id int PRIMARY KEY, events list<int>`). Requests are expected to be SQL strings (in practice `INSERT` and `UPDATE` forms) and are executed deterministically against this table. For `INSERT` requests the implementation parses the `id` and inserts an empty events list; for `UPDATE` requests it parses the event value and appends it to the events list for the given `id`. The implementation uses prepared statements for inserts and updates to avoid injection issues and to ensure consistent CQL.

### Request Handling:

A dedicated processor thread polls ZooKeeper for request nodes, sorts node names to determine order, and iterates nodes while enforcing the invariant that only the next sequence (`lastProcessedSeq + 1`) is applied. If the next sequence is missing (a gap), processing pauses until the missing request appears. After executing each request, the server increments `lastProcessedSeq`, records the node in `processedRequests`, and periodically triggers a checkpoint after a configurable number of operations (every 50 operations by default).

### Checkpointing:

Checkpoints capture a full snapshot of the grade table. When creating a checkpoint the server queries all rows, serializes the number of rows followed by each row's `id` and the size and values of its events list into a binary blob using `ObjectOutputStream`, and writes that blob to ZooKeeper at `/ftdb/checkpoint/<seq>`. The server also writes the checkpoint sequence to `/ftdb/last_seq` so recovery can quickly locate the latest snapshot. This approach makes restore simple and robust at the cost of copying the entire table during each checkpoint.

**Restoration:**

On startup the server reads /ftdb/last\_seq to find the most recent checkpoint sequence. If a checkpoint exists, the server reads the binary snapshot from /ftdb/checkpoint/<seq>, truncates the local grade table, and restores rows from the snapshot using prepared insert statements. After restoring, lastProcessedSeq is set to the checkpoint sequence and normal processing resumes. If no checkpoint is present, the server starts from an empty state with lastProcessedSeq = -1.

**Cleanup and log management:**

To bound state, the server periodically calls cleanupOldRequests() which checks the number of znodes under /ftdb/requests and deletes the oldest ones when the count exceeds MAX\_LOG\_SIZE. Deleted request names are also removed from processedRequests so memory usage stays bounded. This pruning keeps ZooKeeper from growing indefinitely and ensures the server only retains a recent window of pending requests.

**Testing and Results:**

All tests are passed